

CHAPTER 5

SYSTEM DESIGN

5.1 INTRODUCTION TO DEPRESSION DETECTION SYSTEM

The DepHNN Analyzer is a full-stack web application developed for the automated screening of Major Depressive Disorder (MDD) using Electroencephalogram (EEG) signals. It delivers a complete, end-to-end solution, ranging from raw EEG data upload to final diagnostic prediction, all within a secure, user-authenticated web environment.

The primary goal of the system is to provide an objective, data-driven, and accessible tool for mental health screening. Traditional diagnostic methods rely heavily on self-reported symptoms and clinical interviews, which are often subjective and prone to bias. In contrast, DepHNN Analyzer leverages EEG data as a quantitative biomarker of brain activity. By employing advanced deep learning techniques, the system identifies intricate neural patterns that distinguish individuals with MDD from healthy controls.

The design and methodology of the system are based on the research paper *“DepHNN: A Novel Hybrid Neural Network for Electroencephalogram (EEG)-Based Screening of Depression.”* At its core lies a hybrid deep learning architecture that integrates a Convolutional Neural Network (CNN) with a Long Short-Term Memory (LSTM) network. This model is specifically designed to learn directly from raw time-series EEG data, thereby eliminating the need for complex manual feature engineering. The CNN functions as an automatic feature extractor, detecting local spatial patterns across 19 EEG channels, while the LSTM captures the temporal dependencies of these patterns, effectively modeling the evolution of brain activity over time.

To enable practical implementation, the core model is integrated into a robust architecture. Raw .edf EEG files are first processed through a preprocessing pipeline developed using MNE-Python, which performs 0.5–45 Hz bandpass filtering to remove noise, applies Independent Component Analysis (ICA) to correct biological artifacts such as eye blinks and muscle activity, stabilizes the data through Z-score normalization, and segments it into 1-second overlapping windows to prepare it for model input. The backend, developed using Django, manages all user operations through an extended authentication

system, supporting secure registration, login/logout, and One-Time Password (OTP) verification, while hosting the pre-trained TensorFlow/Keras model and exposing it via a protected API endpoint for predictions. The frontend, designed with HTML, CSS, and JavaScript, offers a dynamic and responsive user experience, including a public landing page with “About” and “Contact” sections, as well as a private analyzer dashboard that allows users to upload EEG files, visualize prediction confidence through interactive charts (using Chart.js), and view results clearly and intuitively.

5.2 FEASIBILITY STUDY

This analysis evaluates the practicality, effectiveness, and sustainability of implementing the proposed DepHNN Analyzer web application. It examines the economic, operational, and technical resources required to successfully develop, deploy, and maintain the system. The core objective is to validate that, which replicates, verifies, and deploys a hybrid CNN–LSTM model for depression screening is a viable and scalable solution for both research and clinical demonstration purposes.

5.2.1 ECONOMIC FEASIBILITY

The economic feasibility of the DepHNN is exceptionally high, as both development and deployment rely almost entirely on open-source and freely available technologies. The software stack is completely open-source, encompassing the Python programming language, the Django web framework for backend development, the TensorFlow (Keras) deep learning library, and the MNE-Python library for biomedical signal processing. This approach eliminates all costs associated with proprietary software licenses and development tools.

The dataset employed for model training and validation, the *Figshare “MDD Patients and Healthy Controls EEG Data”*, is publicly available for academic and research purposes, thereby removing any data acquisition costs. The only notable hardware requirement involves a one-time use of a GPU-enabled consumer-grade computer for model training, a resource readily available in most university laboratories or accessible to independent developers.

System deployment is also highly cost-effective. The final Django-based web application, coupled with its lightweight SQLite database, can be hosted on affordable Virtual Private Servers (VPS) or Platform-as-a-Service (PaaS) providers at a minimal monthly cost, keeping operational expenses negligible.

Overall, the project exhibits strong economic feasibility due to its minimal financial requirements, reliance on open-source technologies, and scalable infrastructure, all of which make it a cost-efficient and sustainable solution for research and real-world applications.

5.2.2 OPERATIONAL FEASIBILITY

Operational feasibility assesses the ability of the system to function effectively and be adopted by its target users—clinicians and researchers—with minimal complexity. The DepHNN Analyzer has been designed as a fully automated, end-to-end diagnostic tool requiring no specialized data science expertise from the end-user. The workflow is intuitive, secure, and streamlined. The web application incorporates a robust user authentication system that includes registration, login/logout, and One-Time Password (OTP) verification to ensure data privacy and secure access.

Once logged in, the user—whether a clinician or researcher—can simply upload a raw .edf EEG file through the interface. The system autonomously performs all backend processes, including signal filtering, Independent Components Analysis (ICA) for artifact removal, normalization, segmentation, and deep learning-based prediction. The results are presented in an interpretable and visually engaging format, displaying the diagnostic classification (“Normal,” “Depressed,” or “Major Depressed”) along with a quantitative confidence score and dynamic visualizations generated using Chart.js. This streamlined and automated workflow ensures ease of use while maintaining analytical rigor.

The secure, user-friendly, and fully automated design validates the high operational feasibility of the system, demonstrating its potential for seamless integration into research workflows or as a supplementary clinical screening tool requiring minimal user training. Furthermore, the system’s modular architecture ensures that each component—from preprocessing to prediction—runs reliably without requiring manual intervention. The platform is optimized for low-latency performance, ensuring that even large EEG files are processed and analyzed efficiently. The dashboard layout maintains clarity, allowing users to easily navigate between upload, prediction, and result pages without confusion.

Additionally, the system is designed to be accessible across commonly used devices, making it suitable for deployment in academic labs, hospitals, and remote clinics. Automated error-handling mechanisms guide users with clear messages whenever invalid files or

corrupted data are detected, ensuring smooth operation even for non-technical users. The prediction pipeline consistently produces stable and reproducible outputs, increasing user confidence in the system's reliability. Overall, the seamless interaction between the frontend and backend components enhances the practicality of the system, supporting its long-term adoptability and operational success.

5.2.3 TECHNICAL FEASIBILITY

Technical feasibility determines whether the available technologies and resources are sufficient to develop and implement the system as intended. The successful completion of the project itself serves as empirical proof of its technical feasibility. The system integrates three advanced technological domains—biomedical signal processing, deep learning, and full-stack web development into a single, unified architecture.

In terms of data processing, the MNE-Python library was utilized to perform all required preprocessing on 19-channel, 256 Hz EEG .edf files. The library efficiently executed noise filtering, segmentation, and computationally intensive Independent Components Analysis (ICA) operations in real-time during user interactions.

For model implementation, the TensorFlow (Keras) framework was used to construct and train a six-layer hybrid CNN–LSTM model, achieving a state-of-the-art accuracy of **98.27%** on the custom 19-channel dataset. This performance validates the model's technical soundness and its potential for reliable depression screening.

Within the backend integration, the Django framework proved robust and scalable. It successfully integrated its authentication system—including a custom user model—with the AI pipeline. The backend efficiently loads the pre-trained .keras model into memory upon server startup and executes the complete MNE preprocessing and TensorFlow prediction sequence dynamically for each user request.

The frontend integration, developed in JavaScript, utilizes the Fetch API to securely handle EEG file uploads (with CSRF token validation) and dynamically render diagnostic results and visualizations in real time based on the backend's JSON responses.

The seamless integration of these distinct technologies confirms the project's full technical feasibility. Additionally, the trained model can be optimized using TensorFlow Lite

for deployment on edge devices such as Raspberry Pi or NVIDIA Jetson, which offers promising prospects for future real-time and portable implementations beyond the current project's scope.

5.3 ARCHITECTURES

5.3.1 SYSTEM ARCHITECTURE

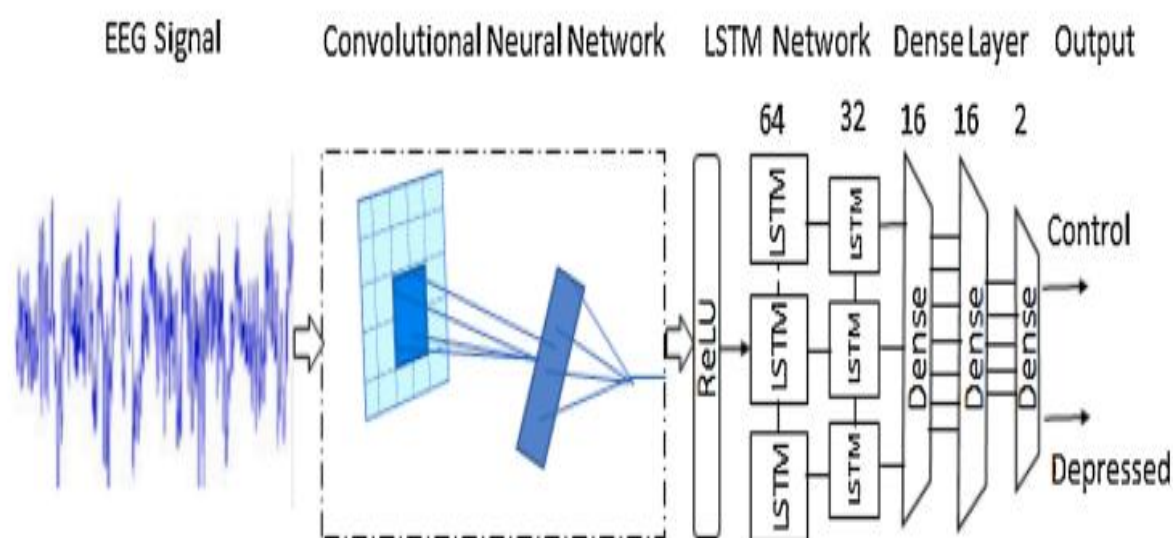


Fig.5.1 System Architecture

The DepHNN (Depression Hybrid Neural Network) is a six-layer hybrid deep learning model combining a Convolutional Neural Network (CNN) with a Long Short-Term Memory (LSTM) network, followed by a dense classifier. The input EEG signal is first segmented into smaller windows during pre-processing, and these windows are fed into the model sequentially.

Layer 1: CNN Feature Extractor uses a 1D convolution with a kernel size of $64 \times 564 \times 5$ to automatically extract local spatial patterns from the raw EEG data. A ReLU activation introduces non-linearity and enhances robustness.

Layers 2 and 3: LSTM Sequence Learner model temporal dependencies in the EEG signals. The first LSTM layer has 64 units, and the second has 32 units, allowing the network to learn long-term dependencies while mitigating the vanishing gradient problem.

Layers 4, 5, and 6: Dense Classifier maps the high-level sequential features to the final output. Layers 4 and 5 are dense layers with 16 units each and ReLU activations, while Layer 6 is a dense layer with 2 units.

The Output layer uses a Softmax activation to produce probabilities for the two classes: "Control" (Normal) and "Depressed," providing a clear and interpretable prediction.

5.3.2 MODEL ARCHITECTURE

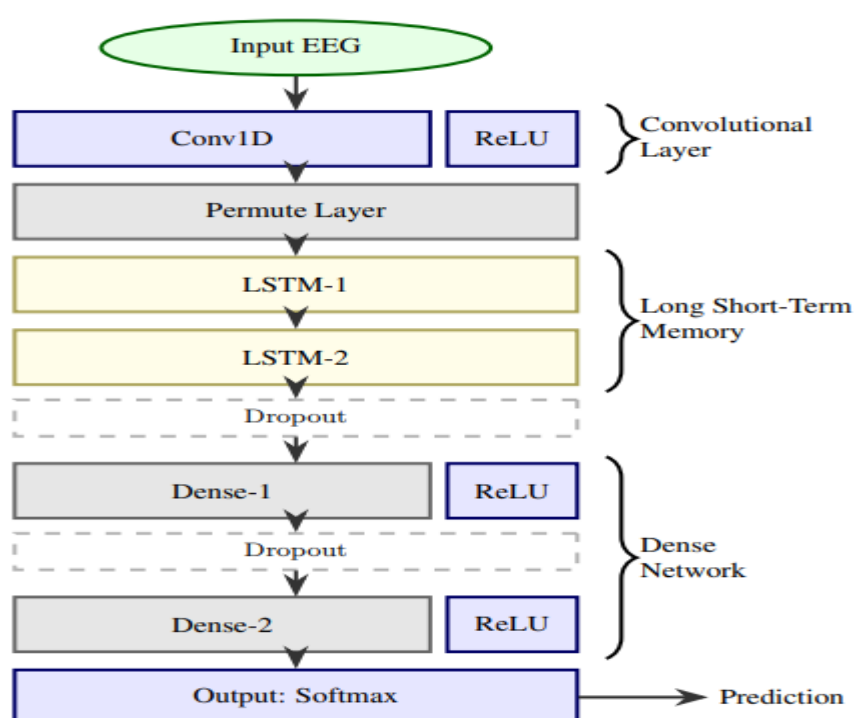


Fig.5.2 Model Architecture

The process begins with raw EEG signals as input. These signals are first passed through a convolutional layer, which extracts important spatial features from the EEG data. The ReLU (Rectified Linear Unit) activation function follows the convolution to introduce non-linearity and help the model learn complex patterns. The extracted features are then passed into two stacked LSTM layers. LSTMs are well-suited for sequential data like EEG because they can retain temporal dependencies and long-term relationships across time. These layers help the model understand how brain signal patterns change over time, which is crucial for detecting depression-related patterns.

After the LSTM layers, the output are fed into a series of fully connected (dense) layers. The first two dense layers are each followed by a ReLU activation function, which helps the network model more abstract and non-linear relationships in the data. The final fully connected layer ends with a Softmax activation, which outputs the probability distribution across different classes for example, "depressed" or "not depressed".

5.3.3 LSTM ARCHITECTURE

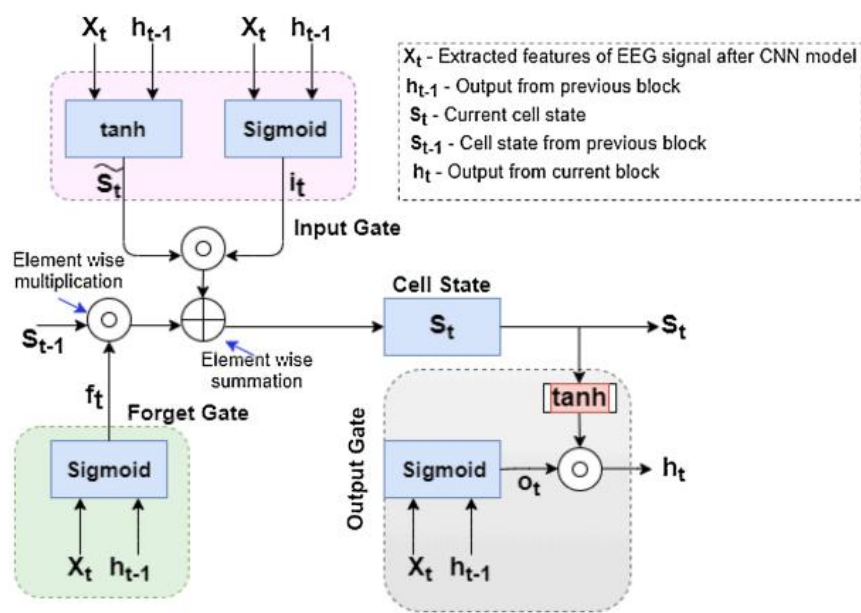


Fig.5.3 LSTM Architecture

An LSTM network processes sequential data such as EEG signals by using a memory cell that can retain information over time. At each time step t , it takes two inputs: the current feature vector X_t (extracted from EEG signals using a CNN) and the previous hidden state h_{t-1} . The forget gate uses a sigmoid function to determine which parts of the previous cell state S_{t-1} should be retained or discarded. The input gate has two components: a sigmoid function that decides which new values to update and a tanh function that generates candidate values to be added to the cell state. These are combined through element-wise multiplication and summation to update the cell state S_t , which acts as the network's long-term memory. The output gate then decides what part of the cell state should be passed to the next layer. It uses a sigmoid function to filter the output and a tanh function to scale the updated cell state, resulting in the new hidden state h_t . This mechanism allows the LSTM to effectively capture

important temporal patterns in EEG data, which is useful for tasks like mental health prediction.

5.4 OBJECT ORIENTED DESIGN

5.4.1 FLOW CHART

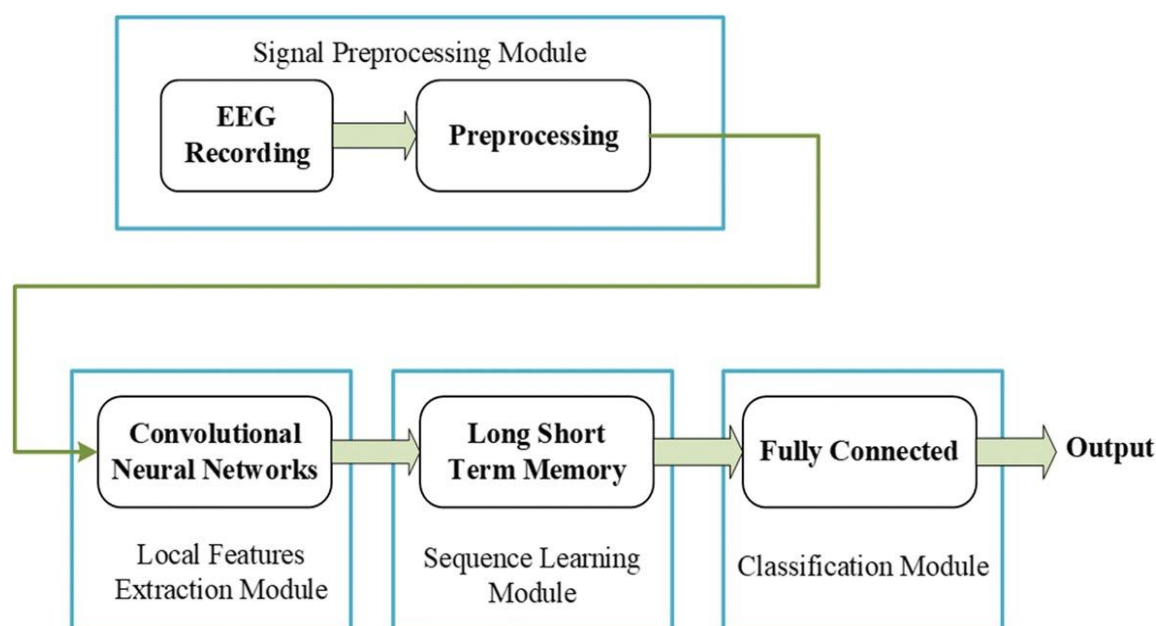


Fig.5.4 Flow Chart

The process starts in the Signal Pre-processing Module, where raw EEG signals are filtered, cleaned of artifacts, and normalized for consistency. These processed signals are passed into a Convolutional Neural Network (CNN), which extracts spatial features from different brain regions. The extracted spatial data is then fed into a Long Short-Term Memory (LSTM) network that captures temporal dependencies, analyzing how brain activity evolves over time—an important factor in identifying depression.

The output from the LSTM is passed to a Fully Connected layer for classification, which predicts whether the subject is depressed or not. This deep learning pipeline removes the need for manual feature engineering and allows the system to automatically learn meaningful patterns. Techniques like dropout and batch normalization are used to improve model generalization and stability. The model is trained using binary cross-entropy loss and optimized with the Adam optimizer for efficient learning.

Multiple EEG segments across various frequency bands are analyzed, and the final output uses a sigmoid activation to produce depression probabilities. This architecture is not only accurate but also scalable and adaptable for other mental health conditions, making it a promising tool in clinical diagnosis and neuropsychiatric research.

5.4.2 USE CASE DIGRAM

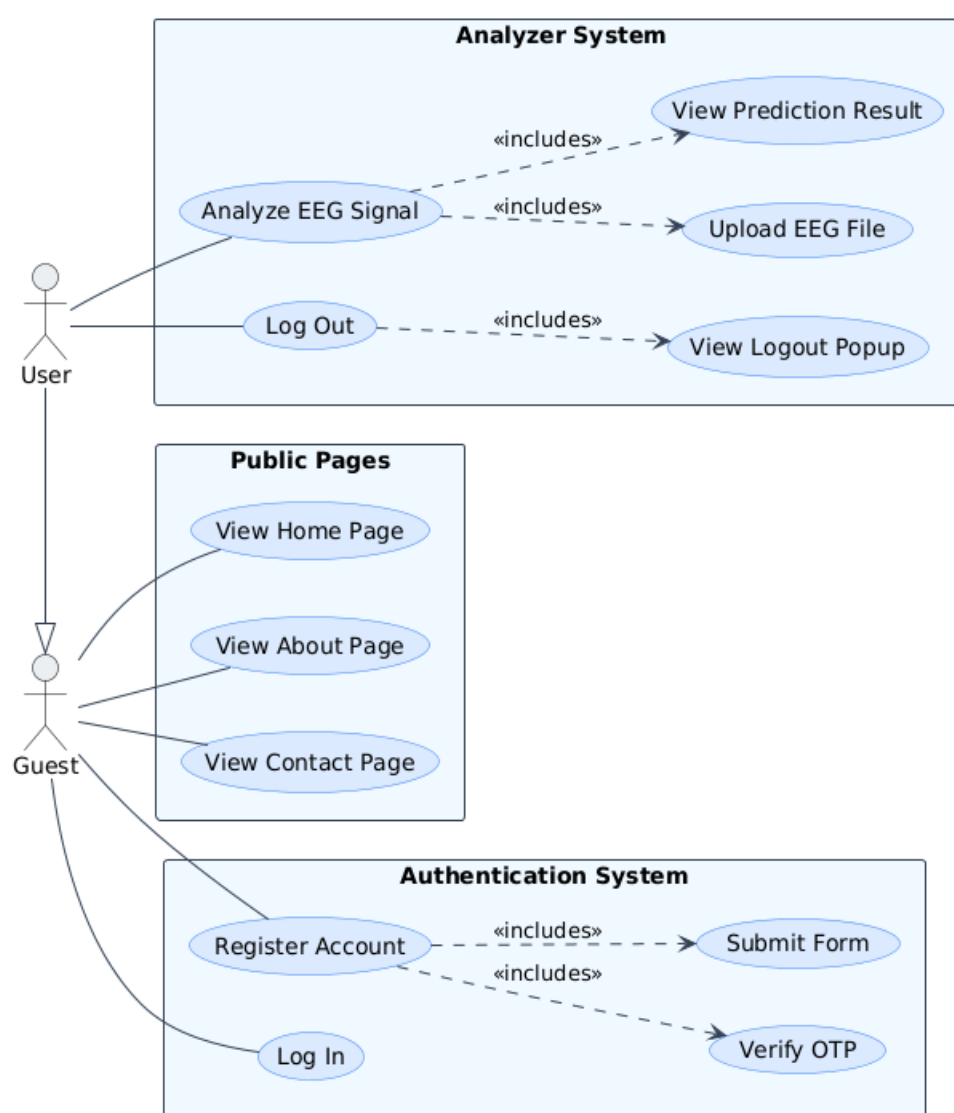


Fig.5.5 Use Case Diagram

The DepHNN Analyzer defines the interactions between its users and core functionalities through two primary actors: Guest, representing an unauthenticated visitor, and **User**, representing an authenticated member. The User inherits all Guest privileges while gaining access to secure system features. Guests can navigate public-facing pages such as Home, About, and Contact, and interact with the Authentication System by either logging in

or registering an account. The registration process is multi-step, encompassing Submit Form, where user details are collected, and Verify OTP, which activates the account. Once authenticated, Users can access the Analyzer System, with the key function being Analyze EEG Signal, including Upload .edf File and View Prediction Result, ensuring that AI-driven analysis remains secure and restricted to authorized members. Additionally, Users can log out, with a Logout Confirmation Popup to prevent accidental session termination, providing both security and usability.

5.4.3 ACTIVITY DIGRAM

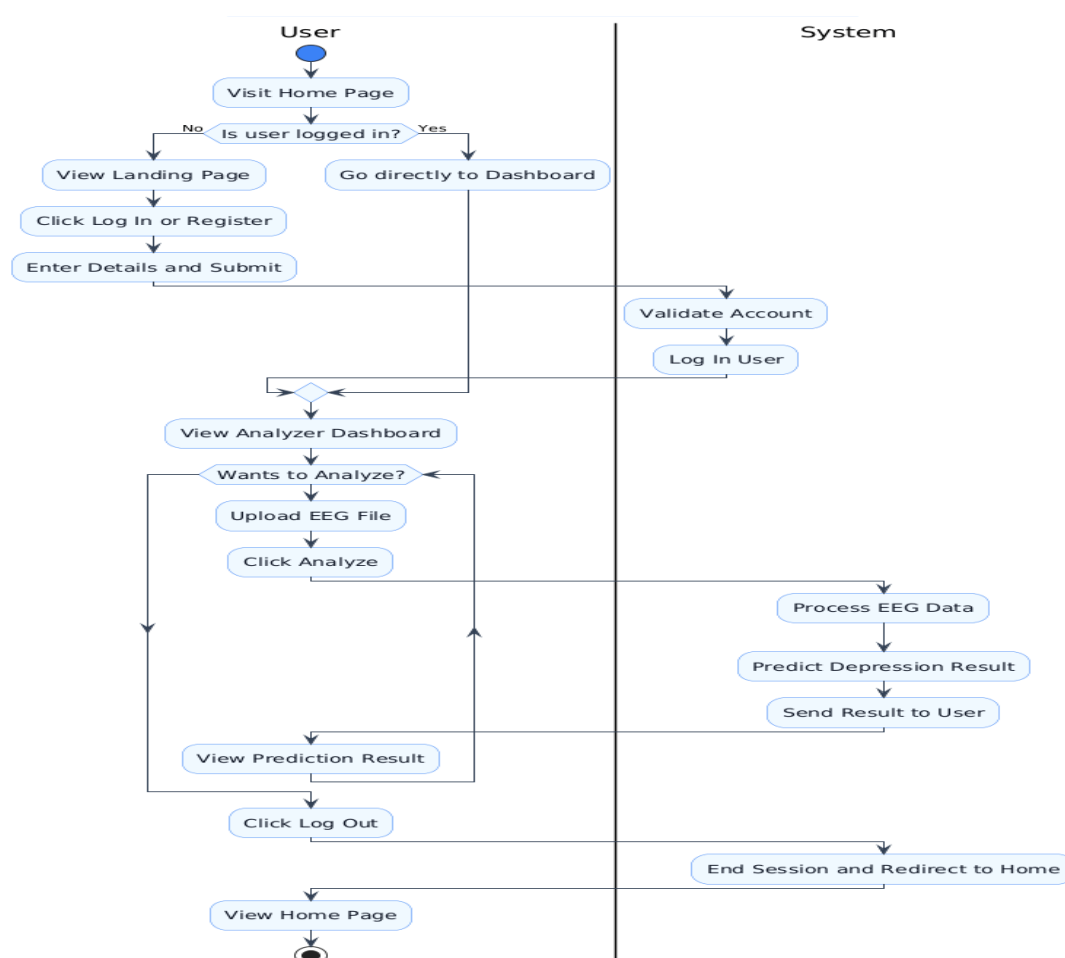


Fig.5.6 Activity Diagram

The workflow of the DepHNN models the sequence of actions and decisions from the user's perspective. The process is organized into two primary roles: the User and the System.

The workflow begins when the User visits the home page. The System first checks whether the user is authenticated. If not, the public landing page is displayed, and the User can choose to Log In or Register. During registration, the System processes the submitted

account details, validates the data, activates the account, and establishes a secure session. Authenticated users bypass this step and are directed to the Analyzer Dashboard.

From the dashboard, the User can enter a loop to analyze EEG data by uploading an .edf file and initiating analysis. The System handles the uploaded file via the /predict/ API endpoint, executes the pre-processing pipeline through `preprocess_single_file()` (including filtering, ICA, normalization, and windowing), and runs the trained CNN-LSTM model using `model.predict()`. The System generates a JSON response containing the diagnostic result and confidence score, which the User's browser interprets to render a Chart.js visualization and display the final prediction. This analysis loop can be repeated as needed.

When the User chooses to log out, a confirmation modal is displayed. Upon confirmation, the System processes the request at the /logout/ endpoint, terminates the session, and redirects the User back to the public landing page, completing the workflow.

5.4.4 SEQUENCE DIGRAM

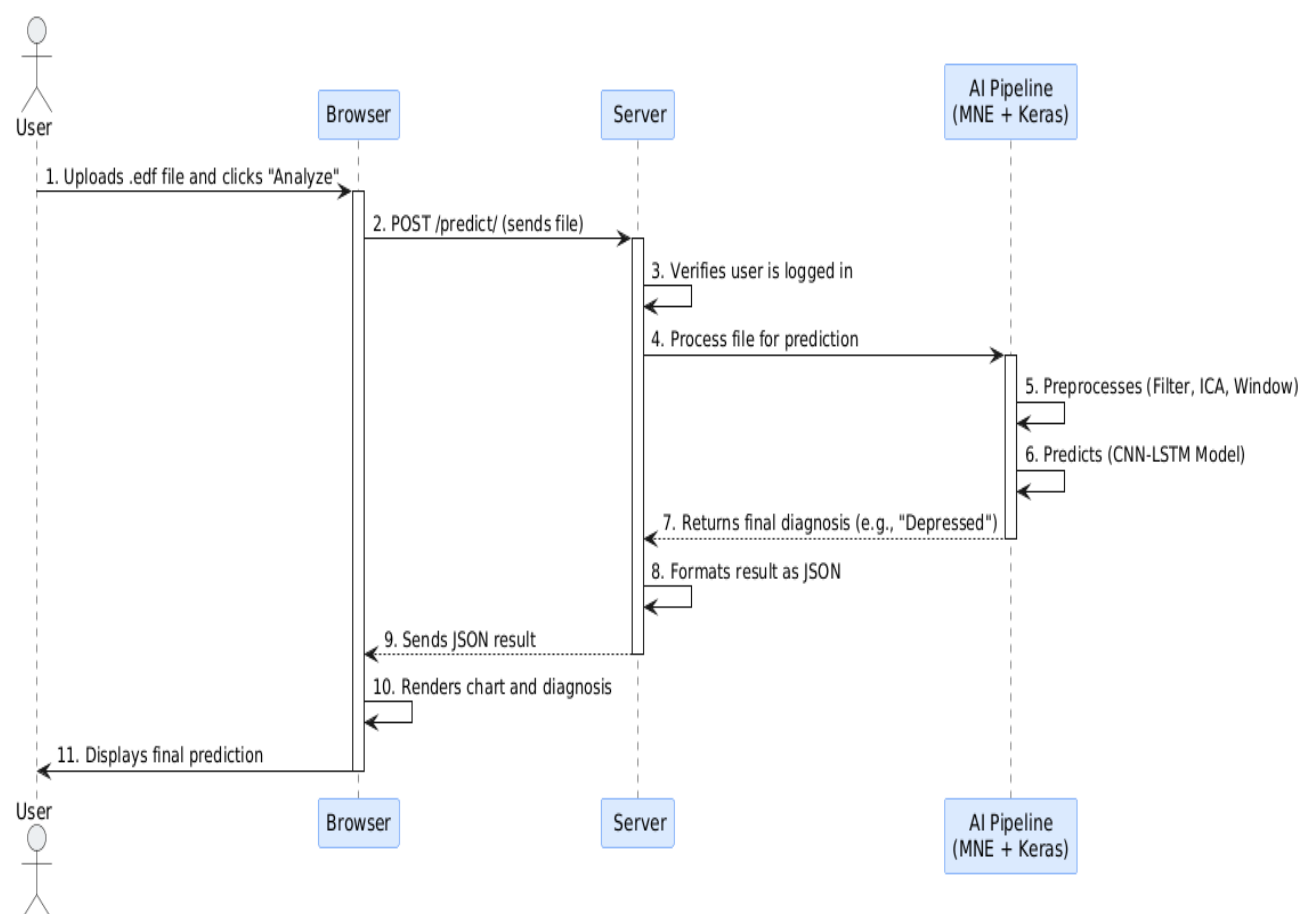


Fig.5.7 Sequence Diagram

The DepHNN Analyzer follows a structured sequence of interactions during a user file prediction, illustrating the chronological flow from the user's action to the final displayed result. Four primary lifelines are involved: the User (actor), the Frontend (Browser), the Backend (Django Server), and the AI Pipeline (MNE + Keras).

The sequence begins when a logged-in User uploads an .edf file and clicks the "Analyze" button. The Frontend's JavaScript packages the file into a FormData object and sends a secure POST request to the /predict/ API endpoint, including the Django CSRF token for security. Upon receiving the request, the Backend first verifies the user's authentication status.

Next, the Backend calls the AI Pipeline in two steps. The preprocess_single_file() function (MNE-Python) loads the EEG file, performs filtering, ICA, and windowing, and returns clean data. This pre-processed data is then fed into the trained Keras model via model.predict() to generate the prediction and confidence scores.

The Backend formats the results into a JSON object and sends it back to the Frontend as an HTTP 200 OK response. The Frontend's JavaScript parses the JSON and invokes the displayResults function, updating the HTML to display the diagnosis and rendering the Chart.js visualization for the User.

CHAPTER 6

IMPLEMENTATION